3 Mixture of Experts

Submission Deadline

This lab must be submitted by **11:59 PM on Friday, 20th March 2026**.

Since we will be moving on to torch and its labs a deadline extension is quite unlikely. You can discuss your doubts more freely on piazza if you are stuck on anything since this lab is going to have absolute grading.

In this graded outlab we will be implementing and exploring a new type of model called a mixture of experts. The explanations here are mostly adapted from Chapter 11 of the Probabilistic ML book. You can refer to that chapter for greater theoretical detail.

3.1 Introduction and Intuitive Explaination

Suppose you wanted to model a probability distribution that behaves differently in different regions of space. One way you might want to do this may be by "combining" various probability distributions. Simply taking a weighted average of the probability densities functions may be fine but it may not give the behavior we want. For e.g. if you are dealing with gaussian random variables then the weighted average will just end up giving you a new gaussian.

One way to model a probability density function is by using a latent variable z which influences the distribution x comes from. So in this case you will be dealing with two distributions.

$$p(z)$$

and

$$p(x|z)$$

You can think of this as follows: You will first draw the latent variable from some distribution (z_i). Then depending upon the value of z_i you will draw x_i from some distribution. Depending on the exact forms of $p(z)$ and $p(x|z)$ you can get many different models here.

3.2 Mixture of Experts

Keep in mind that in the previous section we were trying to model a distribution. Now suppose we are working with a regression or classification task.

Normally we try to model $p(y|x)$. So naturally in this latent variable regime we would like to model

$$p(z|x)$$

and

$$p(y|z, x)$$

So we wish to model two things... What values z likely takes given a particular value of x and based on that how would y be distributed. For this we will take z to be a categorical variable

In this situation $p(z|x)$ is called the **router / gate model**. while $p(y|z,x)$ is called the **expert model**.

Consider the following simple function

$$y = \begin{cases} \sin(x) & \text{if } x < 0 \\ x^3 & \text{if } x \geq 0 \end{cases}$$

Here y behaves very differently for positive and negative values of x so you could imagine that the experts learn the behavior of $\sin(x)$ and x^3 while the router learns to route them appropriately. The exact form of $p(z|x)$ and $p(y|x,z)$ can vary. They can be neural networks, logistic regressor, linear regressor etc. depending on the problem at hand.

We will be training such a model using gradient descent. (There is an alternative way to train such models with "latent / hidden" variables called the **EM Algorithm**. For more details you can refer to the book).

3.3 Good Old Gradient Descent

Let us see how to train such a model using gradient descent. We formally define our model as follows:

$$K : \text{Number of Experts}$$

$$\text{Router: } \boldsymbol{g(x)} : (K,)$$

$$\text{ith Expert: } f_i(\boldsymbol{x})$$

Here we have assumed that we are doing a regression and that y is one dimensional. (You can naturally extend this to multi dimensional y.

So our model output would be as follows:

$$\hat{y} = \sum_{i=1}^{K} \boldsymbol{g(x)}_i f_i(\boldsymbol{x})$$

Here we have just taken a linear combination of the experts weighted by the output of the router. Now we can write the loss function as

$$\mathcal{L} = \sum_{i=1}^{N} (y - \hat{y})^2 / 2N$$

So how do we do gradient descent on this?

$$\frac{\partial \mathcal{L}}{\partial f_i} = \sum_{i=1}^{N} (y - \hat{y}) g_i(\boldsymbol{x}) / 2N$$

You can think of this as doing a gradient descent on f_i where each data-point $\boldsymbol{x}_i$ is weighted by $\boldsymbol{g_i}$.

The only step in backpropagation changed by this is the initial $\boldsymbol{\delta}$ (how?) all other steps follow the same.

You can similarly write an expression for

$$\frac{\partial \mathcal{L}}{\partial g_i}$$

(In that each entry is weighted by $f_i(x)$.)

This similarly only ends up changing the initial $\boldsymbol{\delta}$ for the expert model appropriately.

3.4 Gradient descent for softmax activation

Suppose the final layer of the network produces a vector of logits

$$z = (z_1, z_2, \ldots, z_K) \in \mathbb{R}^K$$

where K is the number of outputs (for example, the number of experts in a router or the number of classes in classification). These logits are transformed into probabilities using the **softmax** activation function

$$s_i = \text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

which produces the vector

$$s = (s_1, s_2, \ldots, s_K) \in \mathbb{R}^K.$$

The output vector s has the following useful properties

$$0 \leq s_i \leq 1$$

$$\sum_{i=1}^{K} s_i = 1$$

so the softmax layer converts arbitrary real-valued logits into a valid probability distribution.

Derivative of softmax Unlike element-wise activations such as ReLU or sigmoid, the softmax function couples all coordinates of the input. Each output s_i depends on every logit z_j. Therefore the derivative is a **Jacobian matrix**

$$J = \frac{\partial s}{\partial z} \in \mathbb{R}^{K \times K}.$$

The entries of this matrix are

$$\frac{\partial s_i}{\partial z_j} = \begin{cases} s_i(1 - s_i) & i = j \\ -s_i s_j & i \neq j \end{cases}$$

which can be written compactly as

$$J = \text{diag}(s) - ss^T.$$

Here

$$\mathrm{diag}(s) \in \mathbb{R}^{K \times K}$$

is a diagonal matrix whose entries are the components of s, and

$$ss^T \in \mathbb{R}^{K \times K}$$

is the outer product of s with itself.

Backpropagating the gradient During backpropagation we receive a gradient with respect to the softmax output

$$g = \frac{\partial L}{\partial s} \in \mathbb{R}^K$$

where L is the loss function.
To propagate the gradient to the logits we multiply by the Jacobian

$$\frac{\partial L}{\partial z} = Jg$$

with

$$J \in \mathbb{R}^{K \times K}, \qquad g \in \mathbb{R}^K.$$

Substituting the Jacobian expression gives

$$\frac{\partial L}{\partial z} = (\mathrm{diag}(s) - ss^T)g.$$

Expanding this matrix multiplication produces a useful simplified form

$$\frac{\partial L}{\partial z} = s \odot \big(g - (s^T g)\big)$$

where

$$s \in \mathbb{R}^K, \qquad g \in \mathbb{R}^K,$$

$$s^T g \in \mathbb{R}$$

is a scalar inner product, and

$$\odot$$

denotes element-wise multiplication.
The final gradient

$$\frac{\partial L}{\partial z} \in \mathbb{R}^K$$

has the same dimensionality as the logits.

From this you should be able to figure out how exactly to update the weights. ($\partial \mathcal{L}/\partial W_{ij}$ should be easy enough to compute from $\partial \mathcal{L}/\partial z$.

You should also update $\boldsymbol{\delta}$ in an appropriate manner for your implementation.
So in the backpropagation function if the activation is softmax then you can do this (notice we bypassed actually computing the Jacobian). For other cases your previous implementation still works.

3.4.1 Entropy Regularization

Since we want our routers to force the experts to specialize. It is beneficial to add an entropy regularization that forces the router outputs to have low entropy (i.e. putting most of the weight on a single expert).We can achieve this by adding an appropriate regularization term as follows

$$\mathcal{L} = \text{MSE} - \lambda * H(\mathbf{g}(\mathbf{x}))$$

Where

$$\text{MSE} = \frac{1}{2N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

$$H(\mathbf{g}(\mathbf{x})) = -\sum_{k=1}^{K} g_k(\mathbf{x}) \log g_k(\mathbf{x})$$

This would change the $\partial\mathcal{L}/\partial g_i$ we calculated earlier but the nice thing about our implementation is that this should just be a few lines added / reomved.

3.5 Task 1

Begin by completing the FFNN.py and build a FFNN implementation that works. This is similar to q2 but you need to extend it in two main ways:

- You need to add support for a softmax final layer.
- Your implementation right now implicitly calculates the gradient of the loss with respect to the output... Instead the backpropogation function takes a precalculated gradient. (that represents $\partial\mathcal{L}/\partial f$ which you can backpropogate appropriately.

This is a somewhat complicated code so its okay if you get stuck / the dimensions messed up. I would recommend trying to reason with specific examples and also be confident of the correctness of each function you write.

We may ask similar codes in the inlabs / exams so please do not use AI to solve this lab.

3.6 Task 2

Now that we have a FFNN architecture we build a MoE class on top of this.

In our class the specifics are as follows:

Router Architecture
The router consists of:

- One hidden layer with ReLU activation
- A softmax output layer which produces the routing probabilities

Expert Architecture

Layer	Operation	Activation
Input Layer	Linear Transformation	–
Hidden Layer	Linear Transformation	ReLU
Output Layer	Linear Transformation	Linear

Each expert follows this same architecture.

In the `forward_pass` function implement the $\hat{y}$ formula we derived earlier:

$$\hat{y} = \sum_{k=1}^{K} g_k(x)\, f_k(x)$$

where $g_k(x)$ is the router output for expert k and $f_k(x)$ is the prediction of expert k.

In the `backward_propagation` function compute the appropriate value of `grad` for each expert and for the router (don't forget to include the regularization term). Then call the `backward_propagation` function of the corresponding networks with these gradients.

If you get mse of less than 0.3 and a significant percentage improvement in moe then your implementation is likely correct

- Compare the number of parameters of your moe model to that of the ffnn and their performance.
- Compare the training epochs taken to get to converge in both cases
- Compare the performance with and without regularization.

Finally you can try to think about the following extensions to this assignment (Please do not make submissions with these extensions implemented. They can be implemented on personal copies)

- Instead of giving all the experts weights in the forward and backward passes. Only give weights to the top k experts. (How would you modify weights to make them sum to 1?... How would this change the gradient expressions?)
- Try to extend this for Experts that are classifiers. Use Binary Cross entropy loss in that case. (How does $\partial\mathcal{L}/\partial f_i$ and $\partial\mathcal{L}/\partial g_i$ change?)

Submission Instructions

Submission will be on moodle. Create a directory named after your roll number, containing the following files:

- `moe.py`

- `ffnn.py`

Then zip the directory into a file named `roll_number.zip` and submit. Write your roll number with all capital letters.